INDU—Individual Assignment in Programming

January 9, 2024

Developed by Kristoffer Sahlin, Lars Arvestad, and Anders Mörtberg.

Contents

General requirements

Basic rules for all project submissions:

- Work individually, i.e., write your own code and find your own solutions.
- Make a note of any help and guidance you have received.
- Program in Python 3.
- Your solution should work without error messages on valid input.

To help with the programming, you can use any module in Python's standard distribution, as well as `matplotlib`, `numpy`, and `scipy`. If there are additional modules you think could be useful, you should ask a teacher first.

Submission

You must submit your code together with a short report on the course website.

The report must be a PDF (`.pdf` file format) to ensure that everyone can read it. The code should be easy to test. Be sure to include any relevant data and test files with your submission.

Report

Your report should not be long. **Maximum length is 3 pages.** You can choose which font, font size, margin size, etc., you want as long as the report is readable.

The report **must** include:

1. Which program and task(s) you have implemented.
2. How the program is started and used.
3. Which libraries/modules are used and how these are downloaded and installed if they are not part of Python's standard distribution.
4. A description of how you structured your program (which files contain what, etc.).

The report **may** also contain reflections on:

- code design,
- which algorithms are used and why,
- which data structures are used and why,
- time and memory efficiency of the code,
- ...

Note: the main purpose of the report is to make it easier for those who will use, test, and evaluate your code! If you do not have a report that contains the four mandatory points above (1.–4.), your project will be rejected.

Oral presentation

After submission, we will randomly select 5-10 students to present their code (just as for labs), so be prepared for this. If you cannot give a detailed and clear presentation of your code (which you should if you wrote it) we will subtract 1-4 points from your score depending on the severity (points and grading scale explained below). Four points corresponds to dropping two grades. You cannot get less than 0 points.

Grading

To pass (grade E), your program must solve the specified task and meet the general (see above) and project-specific requirements. You must include a report as described above.

For higher grades, you must meet our additional quality criteria.

The three criteria that raise your grade are:

1. Good code (3 subcriteria, 2p each, so 6p total).
2. Good error handling (2p).
3. Testing (2p).

These criteria will be scored with 0-2 points according to:

0. Does not meet the criterion.
1. Meets the criterion.
2. Meets the criterion well.

Your grade for the project is determined by how many points you receive (0-10) and is calculated using the following scoring:

Points:	0-2	3-4	5-6	7-8	9-10
Project grade	E	D	C	B	A

Below follows a explanation of our grade-raising critera.

Criterion 1: Good code

To meet this criterion, the code must follow the instructions in the document “Basic principles of programming” and the section on “Good code” in the course compendium. Your code should also be organized appropriately.

The points for this criterion are divided into 3 subcriteria:

A. Documentation (2p)

- An appropriate amount of informative comments that explain the key steps and assumptions in your code.
- Each function should have a documentation string that explains its purpose (beyond what is clear from its identifier).

B. Readability (2p)

- Well-chosen and informative names for identifiers.
- Appropriate line lengths (ideally under 80 characters, definitely under 100).
- Appropriate length of functions (half a laptop screen max).

C. Structure (2p)

- Appropriate division and organization of code into functions so that code duplication is avoided.
- Clear file structure and organization of code.

- Clear separation between functions that do calculations and those that do user interaction. This means your functions for calculations should not contain calls to `print`, `open`, `input`, etc.
- Functions should not depend on global variables, but you can use global constants (that is, variables that never change value while the program is running) if appropriate. If you use any global constants, you should motivate them in your comments.

Criterion 2: Good error handling

The program must not crash on certain runs, neither as a result of random effects that occur in simulations nor because of incorrect parameters or malformed input (either from the user or from a file). Implement appropriate error handling using `try-except` blocks to avoid crashes. You should also raise appropriate exceptions using `raise` when necessary.

In order to meet this criterion well, you should use error handling in a reasonable way as described in the course notes. For example, you should not lift an exception just to capture it directly afterward or use `try-except` when it would be more appropriate to use an `if`-statement.

Criterion 3: Testing

Why should the reviewer trust that the program works? Claiming that "it seems to work" in your report is not enough. You must justify for the reviewer why one can trust that your program works. You should do this by providing a 'test file' that can execute a set of tests for your program. This test file is independent of your program code, but imports your program code as module(s) and tests your functions with suitable values. Object oriented code is not part of this course, but if you are familiar with the concept, feel free to use the library `unittest` for this task. The library `unittest` may help structuring up your tests, but it is not required to use this library.

To fulfill this criterion, you must first write your functions and methods in a way that makes it possible to meaningfully test them (i.e., taking input and returning output is necessary). If you have difficulty writing good tests, it is a sign that you should rethink the design of your program. A good rule of thumb is that by writing short functions that *return* a result, you can much more easily write tests verifying that everything works as intended.